Dependency-Preserving Evidence Packing for Numerical Question Answering over Financial Reports

Pushpendra Singh Badri Bishal Das
Indian Institute of Technology Guwahati
Big Data course project

***Abstract*—A language model answering questions about a financial report sees only the evidence that fits its context window. The answers sit in tables, and a table cell is meaningless without its row and column labels. Dropping the labels makes numbers ambiguous; repeating them with every number costs 2.2 to 8.3 times more tokens per cell and pushes needed numbers out of the window. We represent each report as a dependency graph in which every number links to the labels it needs, and pack evidence so that each label is written once and shared. A PySpark pipeline builds and verifies the graph for six datasets: 1.36 million facts and 1.88 million label links. Over all 9,268 development questions, the method keeps the gold evidence more often than label-repeating packing on all six datasets, by 0.3 to 4.0 points at the chosen budgets. The gain closes as budgets grow and reverses at a 200-token budget. With Gemini 3.5 Flash-Lite, answers are at least as good on all six datasets and better on TAT-QA by 11.3 points.**

I. Introduction

Analysts query annual reports for specific numbers: a margin, a growth rate, a ratio of two line items. Answering such questions is table–text hybrid question answering. A large language model (LLM) can do it, but only from the text in its prompt, and a report is far longer than the evidence budget a system gives the model. The system must retrieve a few facts and *pack* them into that budget.

Tables make this hard. Fig. 1 shows a real FinQA table. The question asks for American Express's average payment volume per transaction. A retriever ranks 637 and 647 highest; only their column headers say that the first is payments volume and the second total volume. Packed without labels, the two are interchangeable. Packed with all their labels, as in serializations such as "the revenue of 2019 is 100" [1], each cell costs several times more and fewer cells fit. We need packing that keeps every number's labels and still fits the budget.

Contributions.

- Dependency-preserving packing: each number is admitted with the labels it points to, and a label already packed costs nothing. We give its exact token trade-off: a fixed cost for structure, then cells 2.2 to 8.3 times cheaper.
- A PySpark data layer that normalises six datasets into one verified store of facts and label links.
- A coverage audit over all 9,268 development questions, five budgets and three packing methods, with no model in the loop.

company	paym. vol.	total vol.	transactions	cards
mastercard	1697	2276	27.0	916
amer. express	637	647	5.0	86
discover	102	119	1.6	57

"What is the average payment volume per transaction for American Express?"

$$637 \div 5.0 = 127.4.$$

Fig. 1. Running example from the FinQA development set (V/2008/page_17, rows abridged). The two top-ranked cells differ only by their column header.

- A paired answer experiment with Gemini 3.5 Flash-Lite, in which the model copies numbers and code checks and computes them.

II. Problem Statement and Objectives

The assigned problem is numerical reasoning over financial reports on FinQA, TAT-QA, ConvFinQA and HybridQA. Our question: *with the same model and the same evidence budget, does keeping each number's headers, units, hierarchy and links improve answers over packing that repeats labels inline?* The hypothesis predicts two limits. Labels consume budget, so the method can lose when almost nothing fits; when the whole document fits, there is nothing to choose. Objectives: (O1) one pipeline over the four assigned datasets plus TAT-HQA and MultiHiertt, scored by the official evaluators; (O2) dependency-preserving packing against label-repeating and length-matched baselines at a fixed budget; (O3) a verified PySpark preparation of all six datasets; (O4) a paired answer test; (O5) answers grounded in the evidence.

III. Datasets

We use the development splits of six datasets (Table I), each from its official repository at a pinned commit, with the supplied document context and the official scorers. FinQA and ConvFinQA share documents, as do TAT-QA and TAT-HQA, so we never pool results. HybridQA is Wikipedia rather than finance and tests whether the method transfers. Passing gold answers from 200 development questions through each scorer gives 100% on five datasets and 90.5% on MultiHiertt, whose span path drops minus signs.

TABLE I
DEVELOPMENT DATA. D AND O: MEDIAN DOCUMENT SIZE IN TOKENS WITH EACH LABEL WRITTEN ONCE AND WITH LABELS REPEATED IN EVERY CELL.

Dataset	Questions	Challenge	D	O	Budget
FinQA	883	programs	1,041	1,112	800
ConvFinQA	1,490 turns	history	979	1,062	800
TAT-QA	1,668	spans, scale	396	592	400
TAT-HQA	824	hypotheticals	396	592	400
HybridQA	3,466	passage links	8,928	11,142	1,500
MultiHiertt	1,044	hierarchy	3,693	6,435	1,500

Sources (GitHub): czyssrs/FinQA, czyssrs/ConvFinQA, NExTplusplus/TAT-QA, NExTplusplus/TAT-HQA, wenhuchen/HybridQA, psunlpgroup/MultiHiertt.



Fig. 2. Pipeline. The highlighted step is the only one that varies.

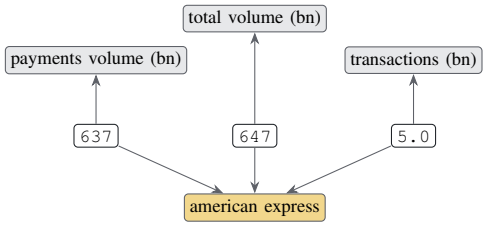


Fig. 3. Dependency graph for three cells of the running example. The row label is a single shared node.

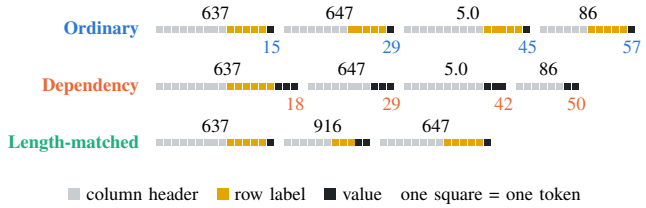


Fig. 4. Packing the American Express row. Numbers below each group are cumulative tokens.

IV. PROPOSED METHODOLOGY

Fig. 2 shows the pipeline. Only the packing step differs between the three methods we compare.

A. Evidence as a dependency graph

Readers turn each document into *facts*: raw text, provenance, and the IDs of the facts needed to read it (Fig. 3). A table cell depends on its column header, row label, corner header and section row. TAT-QA cells also depend on unit paragraphs ("in thousands"); a HybridQA passage depends on the cell that links to it; MultiHiertt cells depend on multi-row HTML headers and section headings. Headers are inferred from layout and flagged uncertain. Gold answers never enter prompts: none of 443 inspected prompts contains a gold label field.

B. Packing under the budget

BM25 ranks cells and paragraphs by the question and earlier questions. Candidates are admitted in rank order as whole bundles; strings are never cut, and a bundle that does not fit is skipped so smaller ones can still enter. Three methods spend the same budget:

- **Ordinary (O)**: each fact carries its structural labels inline. Labels are kept, so the baseline is fair.
- **Dependency (D)**: each fact enters with the recursive closure of its headers, units, hierarchy and link anchors; packed labels are free.
- **Length-matched (L)**: ordinary facts plus their document neighbours, up to D's token count. It tests whether more text alone helps.

Tokens are counted locally on the evidence text as UTF-8 bytes divided by four.

Fig. 4 shows the trade-off. D's first cell costs 18 tokens against O's 15 because its headers come with it; four cells cost 50 against 57. L spends its tokens on a neighbouring Mastercard cell and loses 5.0. Across all development tables, a labelled cell costs 15.1 tokens in FinQA against 3.6 bare (4.2×), 15.6 against 7.0 in HybridQA, and 24.7 against 3.0 in MultiHiertt (8.3×). The median TAT-QA document needs 396 tokens with labels once and 592 with labels repeated; its budget is 400 (Table I).

C. Grounded answers

The model returns a JSON plan of official operations whose arguments are numbers copied from the evidence, e.g. `divide("637","5.0")`. Code locates each number in the packed evidence or the question, rejects the plan if one is missing, and computes the result in a bounded decimal interpreter. Grounding blocks invented numbers but not wrong real ones: `divide("647","5.0")` passes and returns 129.4. Labels are what prevent that choice.

V. BIG DATA IMPLEMENTATION WITH PYSPARK

The six datasets come as JSON (FinQA, TAT-QA, TAT-HQA), a zip archive (ConvFinQA), HTML tables (MultiHiertt) and a linked corpus of 15,314 Wikipedia tables (HybridQA). Python readers normalise them into three JSON-lines tables: documents, keyed by a SHA-256 content hash so repeated contexts are stored once; questions, without labels; and facts, each holding its dependency IDs as an array. Spark 4.3.0 then runs four Spark SQL jobs over these DataFrames (Fig. 5):

- a join of questions to documents;
- an `explode` of each fact's dependency array into an edge table, the graph's links;
- a `left_anti` join of edges against fact IDs, which must be empty;
- a self-join of documents on content hash, which finds identical documents across datasets.

The store is written as Parquet partitioned by dataset and split, so one dataset can be read without scanning the others. The job runs in local mode with 8 shuffle partitions and a 3 GB

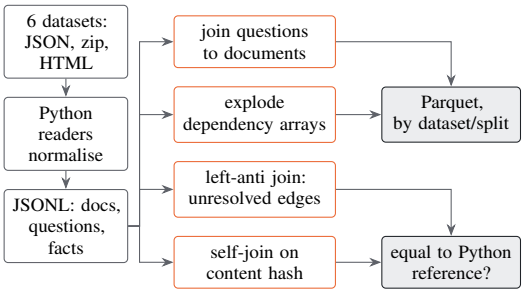


Fig. 5. PySpark data layer. Spark SQL jobs in orange.

TABLE II
WALL TIME (S): SINGLE-PROCESS PYTHON AND SPARK ON 1–8 CORES. SIZE IS RECORDS PER SOURCE DATASET. ALL RUNS EQUAL THE PYTHON REFERENCE.

Size	Facts	Python	1	2	4	8	Peak GB
100	74,655	1.1	27.8	28.4	23.5	23.2	0.9
1000	507,594	6.0	23.4	21.2	20.2	33.4	1.4
full	1,360,644	15.8	54.5	38.4	37.0	65.6	1.7

driver, and fails unless its join count and hash, edge count and overlap list equal those of an independent single-process Python implementation.

Output. The store holds 1,360,644 facts from 9,375 questions over 4,395 deduplicated documents, with 1,884,748 edges and 0 unresolved. The self-join found 491 identical document pairs across datasets, the measured reason we never pool. Parquet uses 0.18 GB against 0.95 GB of JSON lines (5.1× smaller).

Performance. Table II times the same jobs at three data sizes and 1, 2, 4 and 8 cores on an eight-core laptop. On the full data Spark is fastest at four cores, 37.0 s (36,700 facts/s), and slowest at eight, 65.6 s. Single-process Python takes 15.8 s (86,000 facts/s). With 1.4 million rows and a 1.7 GB peak, the data fits in memory, so JVM start-up, shuffles and collecting results to the driver for the equality check are the likely overheads; we did not profile them. Parsing raw files into facts takes 120 s in Python and is the real bottleneck. At this scale Spark contributes a verified, partitioned columnar store, not speed, and we make no cluster scalability claim.

Batch evaluation. The coverage audit runs 139,020 packings (9,268 questions × 5 budgets × 3 methods) over eight worker processes in 679 s. A SQLite ledger stores every model response under a frozen run contract (question IDs, seed, input and code hashes), so runs resume after free-tier quota stops and re-scoring needs no new calls.

VI. EXPERIMENTAL SETUP

Budgets. At 4,000 tokens four datasets were never truncated in a 40-question sample, so the methods could not differ. We set each budget from document sizes, without labels: 800 for FinQA and ConvFinQA and 400 for TAT-QA and TAT-HQA (the largest tested values truncating about 70% of sampled documents), and 1,500 for HybridQA and MultiHiertt, which stay 93% truncated even at 4,000.

TABLE III
MODEL AND INFERENCE CONFIGURATION (FROM THE FROZEN RUN CONTRACTS).

	Block A (packing)	Block B (executor)
Model	Gemini 3.5 Flash-Lite	Gemini 3.1 Flash-Lite
Served version	3.5-flash-lite-07-2026	3.1-flash-lite-05-2026
Training	none (as served)	none (as served)
Output	schema-bound JSON	schema-bound JSON
Max output tokens	2,048	2,048
Thinking level	low	low
Temperature	provider default	provider default
Responses	885	472
Median tokens in / out	1,403 / 129	1,214 / 109
Sample (seed 2626)	50 per dataset	40 per dataset

Measurements. (1) The *coverage audit* checks, with no model, whether every gold operand or answer phrase survives packing; for the running example the gold program is `divide(637, const_5)`, so only 637 must survive. (2) *Block A* answers 50 questions per dataset with each of O, D and L (45 turns in 15 ConvFinQA conversations). (3) *Block B* compares the executor with direct answering on D evidence, 40 questions per dataset.

Model. Table III lists the configuration. Both models are used as served, with no training or fine-tuning; each question is one API call, and the two blocks are never compared with each other.

Scoring. Official execution accuracy for FinQA and ConvFinQA and official F1 elsewhere. Block B uses a 1%-tolerance numeric measure for FinQA and ConvFinQA, since direct answers have no program. Abstained and rejected answers score zero. Intervals are paired bootstrap resamples of whole documents (2,000 resamples, seed 2626), uncorrected across the six comparisons per block.

VII. RESULTS

A. Evidence coverage

At the chosen budgets D keeps more gold evidence than O on all six datasets (Table IV, Fig. 6). Questions kept by only one method favour D 146 to 7 on HybridQA and 42 to 3 on MultiHiertt; FinQA (9 to 1) is weaker and ConvFinQA (6 to 1) is not significant. The gap is largest where documents exceed the budget most, and smallest on FinQA and ConvFinQA, where labels add only 7–8% to a document. It shrinks as the budget grows and is zero at 4,000 tokens on the four smaller datasets. At 200 tokens the fixed cost of structure loses: -1.8 points on ConvFinQA (36 against 63 discordant) and -0.8 on MultiHiertt (50 against 58). L is below O at every budget, so extra neighbouring text does not substitute for labels.

B. Effect on answers

With Gemini 3.5 Flash-Lite, D scores at least as high as O on all six datasets, by 0 to 11.3 points (Table V). Only TAT-QA's gain has an interval excluding zero: +11.3 [+4.1, +19.9]. MultiHiertt (+9.8) and FinQA (+2.0) touch zero at the lower bound. The pattern follows coverage: largest on hierarchical tables, absent on ConvFinQA, whose documents nearly fit.

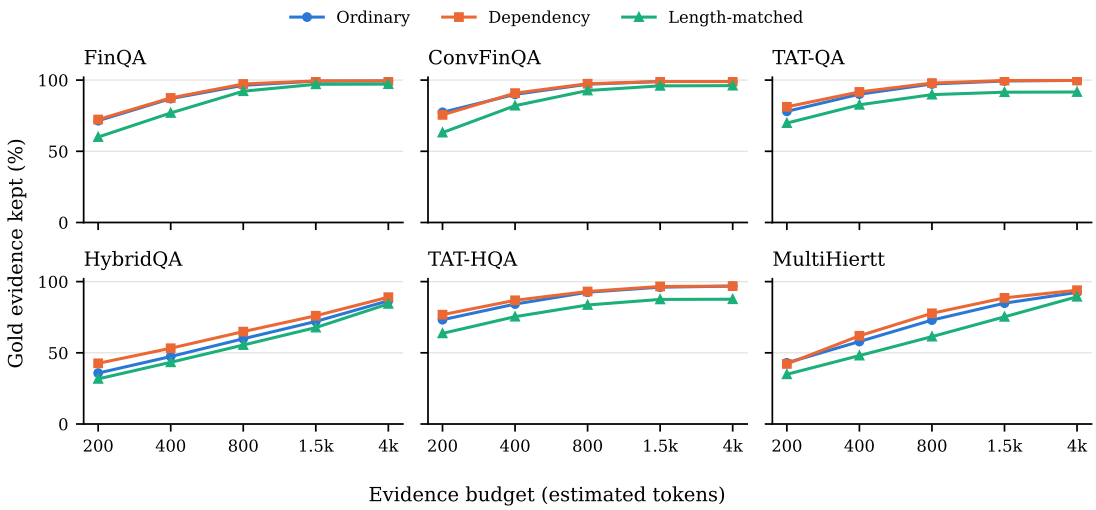


Fig. 6. Gold evidence retained against budget, full development splits.

TABLE IV
QUESTIONS WITH ALL GOLD EVIDENCE RETAINED (%). D+/O+: KEPT ONLY BY D OR ONLY BY O; p: TWO-SIDED SIGN TEST.

Dataset	n	Cap	O	D	L	D+/O+	p
FinQA	845	800	96.3	97.3	92.2	9/1	0.021
ConvFinQA	1486	800	97.1	97.4	92.7	6/1	0.125
TAT-QA	1636	400	90.1	91.7	82.6	41/15	<.001
HybridQA	3466	1500	72.0	76.0	67.7	146/7	<.001
TAT-HQA	792	400	84.2	86.9	75.4	33/12	0.002
MultiHiertt	1043	1500	84.9	88.6	75.3	42/3	<.001

TABLE V
BLOCK A ANSWER SCORES (%), GEMINI 3.5 FLASH-LITE. EXECUTION ACCURACY FOR FINQA AND CONVFINQA, F1 OTHERWISE.

Dataset	n	O	D	L	D−O	95% CI
FinQA	50	62.0	64.0	66.0	+2.0	[+0.0, +6.2]
ConvFinQA	45	73.3	73.3	68.9	+0.0	[−6.0, +6.4]
TAT-QA	50	60.2	71.5	69.4	+11.3	[+4.1, +19.9]
HybridQA	50	54.9	60.1	49.8	+5.3	[−2.4, +13.0]
TAT-HQA	50	56.7	60.7	52.7	+4.0	[−6.4, +14.5]
MultiHiertt	50	47.0	56.8	41.0	+9.8	[+0.0, +20.0]

TABLE VI
BLOCK B (%), GEMINI 3.1 FLASH-LITE: EXECUTOR AGAINST DIRECT ANSWERING ON D EVIDENCE. MULTIHIERTT IS OFFICIAL F1; UNDER ONE NUMERIC RULE THE GAP IS +6.2.

Dataset	n	Executor	Direct	Diff.	95% CI
FinQA	40	62.5	67.5	−5.0	[−17.5, +7.0]
ConvFinQA	36	75.0	75.0	+0.0	[−8.6, +7.3]
TAT-QA	40	65.2	58.7	+6.5	[−7.6, +20.7]
HybridQA	40	50.2	66.6	−16.4	[−30.6, −2.0]
TAT-HQA	40	60.0	56.7	+3.3	[−12.0, +19.3]
MultiHiertt	40	63.7	41.7	+22.1	[+2.5, +40.0]

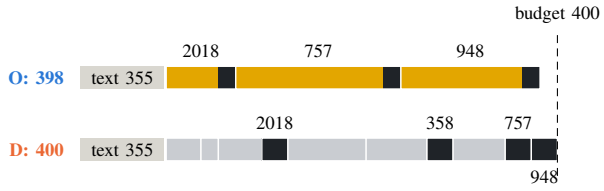


Fig. 7. TAT-QA question at budget 400. Yellow: labels inline; dark: values; grey: labels written once. O (398 tokens) has no room for 358; D (400) fits it.

Fig. 7 replays one TAT-QA question with the saved model outputs: the ratio of inventory reserves to accrued compensation in 2018, at a budget of 400. Both methods pack the same two paragraphs (355 tokens). O fills the rest with three labelled cells and stops at 398; the accrued-compensation cell, 358, needs 17 more tokens. The model reports that the evidence lacks accrued compensation and abstains. D pays 33 tokens for the labels once and 3 per cell, fits 358, and the model's plan `divide("948","358")` returns 2.65, the gold answer.

C. Executor against direct answering

With Gemini 3.1 Flash-Lite, the executor gives no consistent gain (Table VI). Direct answering leads on HybridQA by 16.4 points, with an interval excluding zero; its answers are short text spans with no arithmetic. The executor's 22.1-point lead on MultiHiertt is a scoring artefact: direct answers write percentages (−29.92 for a gold −0.2992), which the official span path marks wrong. Under one numeric rule for both sides the difference is +6.2 [−6.2, +20.0], so we make no claim. The executor's value is that every answer traces to specific evidence. Most remaining errors are wrong values, such as reversed percentage changes, or wrong or over-long spans.

VIII. LIMITATIONS

Coverage measures presence, not correct use. Samples of 50 leave most per-dataset answer gains within noise. Headers are inferred and can be wrong; budgets are estimated tokens; BM25 misses paraphrases. Hosted models can change between runs. ConvFinQA and HybridQA test labels are not public, so all results are on development splits.

IX. Conclusion

Large tables must fit a model's fixed context window without losing what their numbers mean. Writing each label once and linking every number to it keeps more of the evidence a question needs at the same budget: on all six datasets at the chosen budgets, most where documents exceed the budget, with a measured loss at 200 tokens. Answers from Gemini 3.5 Flash-Lite follow, with a significant gain on TAT-QA. PySpark builds and verifies the 1.36-million-fact graph the method runs on. Next steps are larger answer samples, budget-aware closure that pays for a header only when several packed facts share it, and better header inference for nested tables.

References

[1] A. Arun, A. Dhiman, M. Soni, and Y. Hu, "Numerical reasoning for financial reports," *arXiv preprint arXiv:2312.14870*, 2023.